

电脑的重量与尺寸依然限制了人们使用计算机的自由度，基于键盘、鼠标（含触点杆和触摸板）的交互方式也阻碍了计算机的进一步普及。与此同时，随着移动通信网络技术的不断发展，高速无线数据传输的支持使得无处不在的互联网访问成为可能，这开启了移动互联网时代。

2007 年，Steve Jobs 发布了第一代 iPhone，同样标志着 iOS 操作系统的诞生。与之前的智能手机不同的是，iPhone 采用触摸屏的交互方式，从而支持更加自然的人机交互。此外，iOS 也在应用与应用模式上具有显著的创新，新的 AppStore 一方面提供了移动应用分发与销售的新模式，另一方面也为移动生态的构建提供了新的基础。

图 1.5 显示了 iOS 与 macOS 的架构分层及异同。iOS 的设计在很大程度上复用了 macOS 的功能，包括 XNU 操作系统内核（由 Mach 微内核和 BSD 内核叠加构成的混合内核），以及一些核心 OS 服务（Core OS Services），并将 macOS 的应用程序框架（Cocoa）针对 iPhone 等触屏设备进行了重新设计与优化，形成了新的面向智能手机的应用程序框架 Cocoa Touch。值得一提的是，苹果公司面向其他设备的操作系统如 iPadOS、tvOS 与 watchOS，也是根据设备的特点从 iOS 衍生出来的。这些不同的操作系统都通过 iCloud 提供云服务，例如账号管理、存储、应用分发、支付等，从而带来跨多设备的一致性体验。

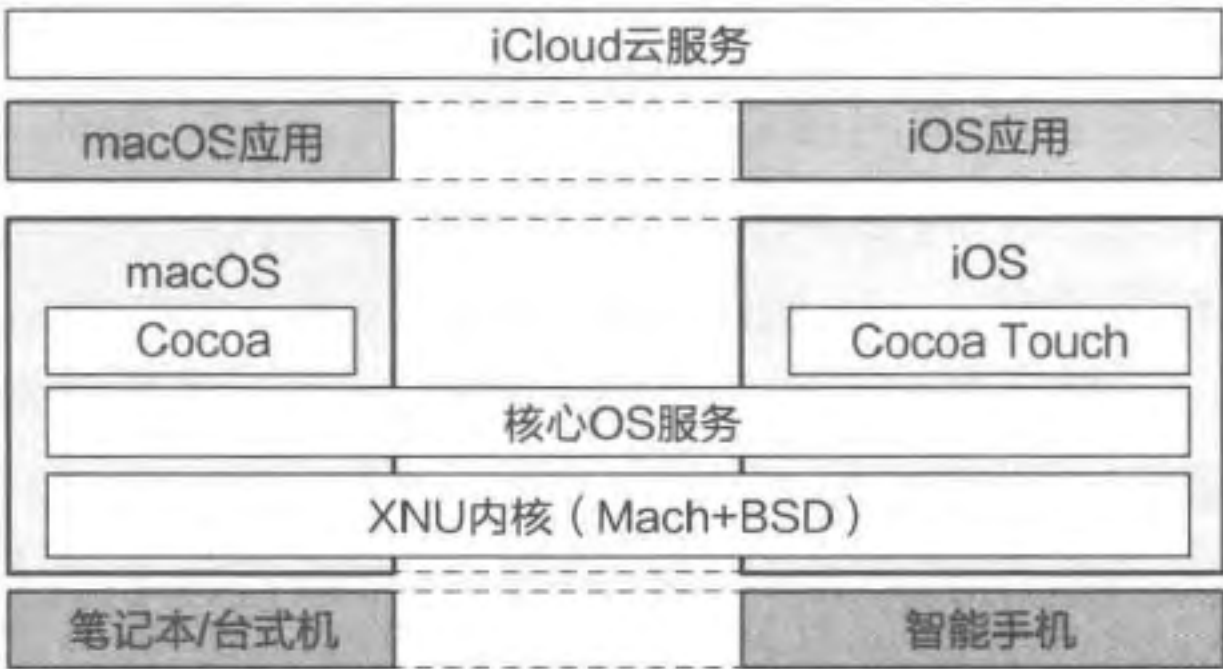


图 1.5 iOS 与 macOS 的架构分层及异同

移动互联网时代的来临也促使谷歌公司投入移动操作系统的研发，从而抢占新时代的入口（如移动搜索和移动广告）。谷歌公司在 2005 年收购了 Andy Rubin 主导设计的 Android 操作系统，并在此基础上发布了 Android 1.0 操作系统。在图 1.1 中，我们已经展示了 Android 操作系统的基本架构。不同于 iOS 采用核心代码闭源、生态强管控的方式，Android 采用开源的方式来构建生态。2007 年 11 月，谷歌公司联合 34 家芯片制造商、网络运营商、手机设备生产商与应用开发者成立开放手机联盟（Open Handset Alliance, OHA），从而使 Android 逐步在移动互联网时代的操作系统中占据了主流地位。

此外，由于移动应用开发越来越多地需要使用云服务提供的能力，诞生了移动后端即服务（mobile Backend as a Service, mBaaS）模式：操作系统将应用开发所需要的一些通用的后端功能（如账号管理、推送通知、社交服务等）汇聚成一个服务集合，以 SDK 和 API 的形式提供给移动应用开发者。例如，苹果公司提供了 CloudKit 以方便应用开发，谷歌公司也基于其收购的 Firebase 构建了谷歌移动服务（Google Mobile Services, GMS）。由于 CloudKit 和 GMS 沉淀了很多应用的共性功能并提供了云服务的入口，它们也成为重要的生态入口与控制点。

1.4 操作系统接口

操作系统在几十年的演进过程中形成了一些相对稳定的接口，这些接口又沉淀到不同的层次。越是下层的接口，数量相对更少，变化相对不频繁；越是上层的接口，数量相对更多，变化相对更频繁。图 1.6 展示了不同层次的操作系统接口，主要包括系统调用接口、POSIX 接口、领域应用接口。

系统调用接口。应用程序通过操作系统内核提供的接口向内核申请服务，这些接口通常称为系统调用接口。不同的操作系统提供的系统调用接口往往各不相同，同一操作系统的不同版本所提供的系统调用接口也会有所变化。

图 1.7 以 hello 程序中的 `printf` 为例展示了系统调用的执行过程。首先，`printf` 调用标准库 `libc` 中的 `write` 函数。`libc` 在准备好相关的参数后，执行 `svc` 指令^①使控制流从应用程序的代码转移到操作系统内核的代码，即运行主体由应用程序切换为操作系统内核。内核的处理函数根据系统调用传入的第一个参数，识别出该调用需要执行操作系统内核提供的 `sys_write` 函数，从而通过系统调用表找到并调用该函数。从上述例子可以看出，系统调用是用户态应用向操作系统内核请求服务的方法。

POSIX 接口。由于每个操作系统提供的系统调用各不相同，为了实现同一应用程序在不同操作系统上的可移植性，逐渐形成了关于操作系统接口的一些标准，POSIX 是其中应用最广泛的一个。POSIX 是 Portable Operating System Interface for uniX 的简写，即可移植操作系统接口，X 表明其是对 UNIX API 的传承。在 POSIX 被提出之前，世界上存在很多不同的 UNIX 操作系统，如 FreeBSD、UNIX System V、Solaris、NetBSD 等。这些接口各异的操作系统对应用程序开发人员造成了比较大的困扰。为了使应用程序能够运行在不同的 UNIX 操作系统之上，IEEE 于 20 世纪 80 年代定义了一套标准的操作系统 API，其正式名称为 IEEE Std 1003，国际标准名称为 ISO/IEC 9945。POSIX 这个名称是由开源软件先驱 Richard Stallman 应 IEEE 的请求而取的一个易于记忆的名称。POSIX 标准通常通过 C 语言库（C library，`libc`）在用户地址空间实现，常见的 `libc` 包括 `glibc`、`musl`、`eglibc` 等，Android 也实现了一个名为 `bionic` 的 `libc`。通常而言，应用程序只需要调用 `libc` 提供的接口就可以实现对操作系统功能的调用，这样一方面可支持应用在类 UNIX 系统（包括 Linux）上的可移植性，另一方面也使新的操作系统可以通过移植 `libc` 支持现有的应用生态。



图 1.6 不同层次的操作系统接口

① `svc` 指令是一条 ARM 指令，表示系统调用（Supervisor Call）。

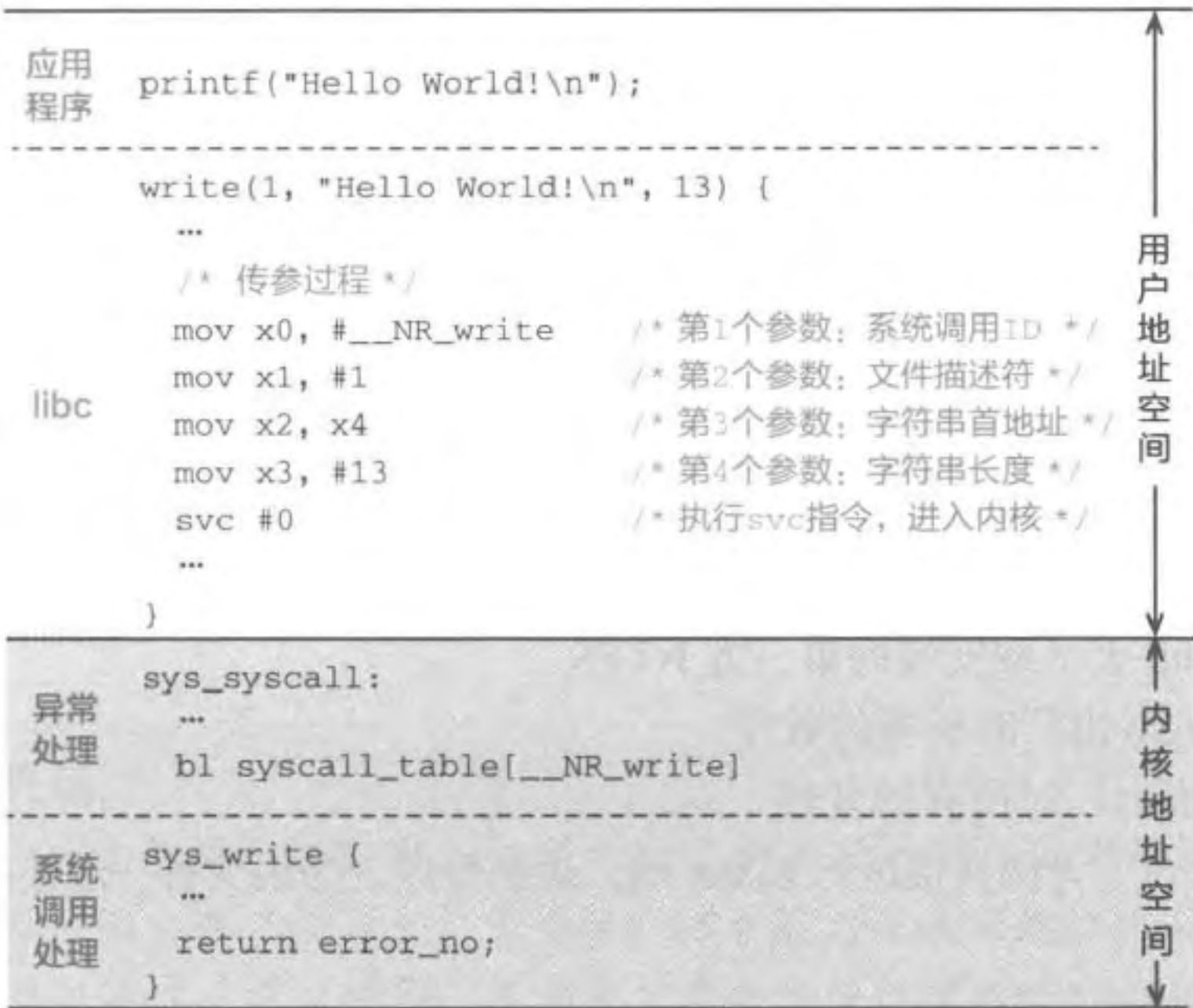


图 1.7 典型的系统调用过程示例

领域应用接口。在 POSIX 或操作系统调用的基础上还可以封装面向不同领域的应用接口。为了应用开发的便捷性（如更多可复用的功能），人们逐渐开始为各个应用领域定义应用开发接口与软件架构。例如，面向汽车领域，一些车企联合起来定义了 AUTOSAR（AUTomotive Open System ARchitecture），从而方便汽车电子平台各个部件的开发者遵循同一标准和软件架构进行开发。随着汽车智能化引起的功能需求的增加，AUTOSAR 也逐步演进到了 Adaptive AUTOSAR，并提供更为丰富的应用开发接口。针对移动平台，Android 应用框架为移动应用开发人员在 Android 操作系统上开发 App 定义了应用开发接口，iOS 同样也为苹果手机平台定义了应用开发接口。此外，前文提到的 CloudKit、GMS 等后端服务也为应用提供了访问云服务的应用开发接口。

API 与 ABI

我们经常听到各种兼容，例如 POSIX API 兼容与 Linux ABI 兼容等。API 是指应用编程接口（Application Programming Interface），其定义了两层软件（例如 libc 与 Linux 内核）在源码层面上的交互接口。ABI 是指应用二进制接口，即在某个特定体系结构下，两层软件在二进制层面上的交互接口，包括如何定义二进制文件格式 [如 ELF（Executable and Linkable Format）格式或 Windows 中的 EXE 文件格式]、应用之间的调用约定（包括参数传递与返回值处理）、数据模式（大端模式、小端模式）等。

可以看到，“层次”在操作系统中无处不在，这是操作系统应对复杂性的一种常见方法。在进一步介绍操作系统每个功能的具体实现之前，下一章将先从宏观的层面介绍操作系统为了应对复杂性，在演进过程中产生的不同架构类型，以及这些架构的优缺点。

1.5 思考题

1. 以下哪些属于操作系统？
 - (a) Windows 10 所包含的所有软件
 - (b) Linux 内核以及所有设备的驱动
 - (c) 在 MacBook 上下载安装的第三方 NTFS
 - (d) 华为 Mate 30 出厂时所有的软件
 - (e) 大疆无人机出厂时所有的软件
2. 在 Windows 平台上无法直接运行 Linux 的二进制程序，是因为哪一层接口不同导致的？
 - (a) ISA 不同
 - (b) 系统调用不同
 - (c) ABI 不同
 - (d) API 不同

参考文献

- [1] RYCKMAN G F. The IBM 701 computer at the general motors research laboratories [C] // Classic Operating Systems Springer, 2001: 37-40.
- [2] BROOKS F P, Jr. The mythical man-month: essays on software engineering [M]. New York: Pearson Education, 1995.
- [3] VYSSOTSKY V A, CORBATÓ F J, GRAHAM R M. Structure of the multics supervisor [C] // Proceedings of the November 30-December 1, 1965, fall joint computer conference, part I. 1965: 203-212.
- [4] RITCHIE D M. The evolution of the unix time-sharing system [C] // Symposium on Language Design and Programming Methodology. Springer, 1979: 25-35.

操作系统概述：扫码反馈

